

Start with a Testable User Need

A requirement is a testable statement of needed behaviour or constraint.



CORE CONCEPT

Architecture Assigns Responsibilities to Components

An architecture identifies components, responsibilities, interfaces, and constraints. A capstone may include: Use the fewest components that satisfy requirements. Every additional service adds failure and cost.

web or API client;

request validation;

workflow state;

retrieval and vector store;

model provider;

structured output;

RUN IT AND INSPECT THE RESULT

Data Flow Reveals Trust Boundaries



```
[User] --untrusted question--> [API validation]  
[Retriever] --authorized chunks--> [Model]  
[Model] --untrusted draft--> [Output validation]
```

OUTPUT

Every boundary has an owner and required check.

Policy documents are approved internally, but their extracted text may contain parser errors. User question is untrusted. Model output is untrusted. Final response is released only after schema, citation, and authorization checks.

REASONING SEQUENCE

Select Models from Evaluation Evidence

Frame

baseline;

1

Transform

at least two suitable
candidates;

2

Validate

quality on the same
test cases;

3

Explain

structured-output and
tool capability;

4

Candidate A: groundedness 0.97, p95 4.5s, cost \$0.03 Candidate B: groundedness 0.95, p95 2.2s, cost \$0.01 If minimum groundedness is 0.95, p95 maximum 3s, and cost maximum \$0.02, only B passes all constraints.

CORE CONCEPT

Use Structured Output, RAG, and Tools Only When Needed

Use structured output when downstream code needs fields and types. Use RAG when answers require current, private, or cited evidence. Use tools when the system must retrieve live data or perform a bounded operation.

structured output: answer, status, citation IDs;

RAG: approved handbook;

tool: read-only ticket lookup only if ticket context is required;

no write-capable agent because the requirement is answer support.

RUN IT AND INSPECT THE RESULT

Workflow State Makes Behaviour Inspectable

```
class CapstoneState(TypedDict):  
    request_id: str  
    evidence: list[Document]  
    attempts: int  
    cost: float  
    status: str
```

OUTPUT

A state record sufficient to explain every route and final outcome.

validate -> retrieve

CORE CONCEPT

The API Contract Includes Failure Responses

Define: Clients must distinguish supported answer, insufficient evidence, denied access, invalid input, and unavailable service.

request schema;

response schema;

authentication;

authorization;

status codes;

idempotency where actions occur;

REASONING SEQUENCE

Evaluation Covers Components and User Outcomes

Frame

request validation;

1

Transform

retrieval precision
and recall;

2

Validate

groundedness and
citation support;

3

Explain

structured-output
validity;

4

Release thresholds:

RUN IT AND INSPECT THE RESULT

Observability Connects a Result to Its Causes

```
trace.set_attributes({  
  "model_version": MODEL_VERSION,  
  "corpus_version": CORPUS_VERSION,  
  "final_status": state["status"],  
})
```

OUTPUT

A trace that can reproduce the component path behind one result.

Total 1,805ms; generation owns most latency.

QUALITY GATE

Security and Cost Are Release Requirements



scoped identities and secrets;



tenant-aware retrieval;



input and output validation;



tool allowlists and argument checks;

RISK REVIEW

Release Includes Rollback and Failure Demonstrations

01 **malformed request;**

02 **unsupported question;**

03 **unauthorized access;**

GUIDED LAB

Capstone Deliverable: Evidence before Presentation

01

**problem statement
and acceptance
criteria**

02

**architecture and
data-flow
diagrams**

03

**threat model and
cost estimate**

04

**versioned source,
configuration, and
environment**